Enhancing Scientific Reproducibility in CrocoDash for Regional Ocean Modeling

Andrew Kwong

Mentors: Manish Venumuddula, Alper Altuntas, Dan Amrhein

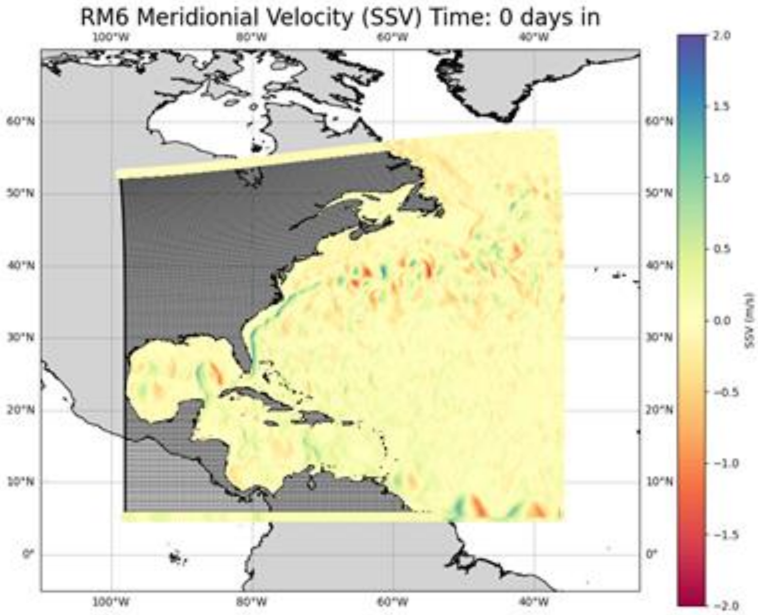
Summer Internships in Parallel Computational Science (SIParCS)

November 7, 2025

This material is based upon work supported by the NSF National Center for Atmospheric Research, which is a major facility sponsored by the U.S. National Science Foundation under Cooperative Agreement No. 1852977.

Regional ocean models are **powerful** tools.

It can be time-consuming to set these models up.



Months/Weeks → Days/Hours

PROJECT OBJECTIVE

Enhance **scientific reproducibility** and increase **accessibility** in the CrocoDash workflow.

Goal: Fully reproducible end-to-end workflow to create regional Modular Ocean Model version 6 (MOM6) cases!

Case: A specific instance of a model simulation in the Community Earth System Model (CESM).

Domain: Specific region being simulated, including a horizontal grid, vertical grid, and topography.

Forcings: Initial conditions (ICs), open boundary conditions (OBCs), tides, and chlorophyll.

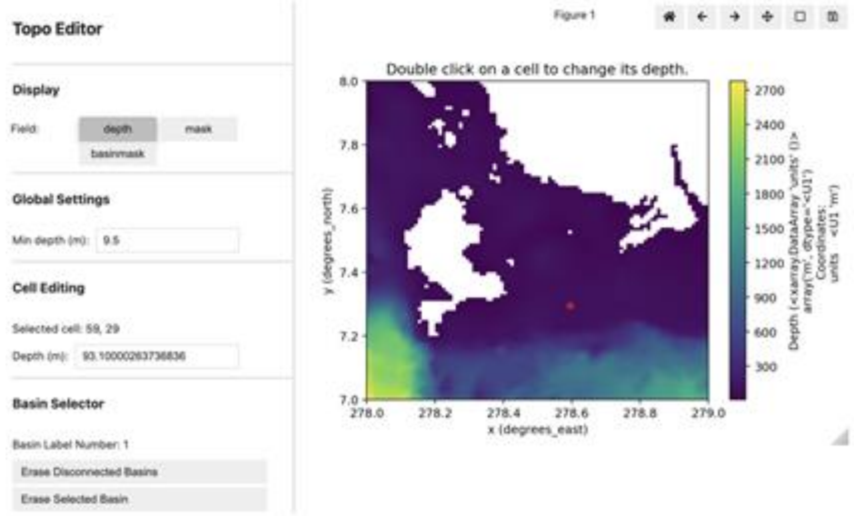
CrocoDash

Domain Generation → Case Creation → Ocean Forcing Data Preparation → Build and Run Case

MOTIVATION

Model setups are now more user-friendly and streamlined in this workflow but…

There is a lack of **reproducible** features in CrocoDash!



SECTION 1: Generate a regional MOM6 domain

We begin by defining a regional MOM6 domain using CrocoDash. To do so, we first generate a horizontal grid. We then generate the topography by remapping an existing bathymetric dataset to our horizontal grid. Finally, we define a vertical grid.

Step 1.1: Horizontal Grid

```
from CrocoDash.grid import Grid

grid = Grid(
  resolution = 0.01,
  xstart = 278.0,
  lenx = 1.0,
  ystart = 7.0,
  leny = 1.0,
  name = "panama1",
)
```

Step 1.3: Vertical Grid

```
from CrocoDash.vgrid import VGrid

vgrid  = VGrid.hyperbolic(
    nk = 75,
    depth = topo.max_depth,
    ratio=20.0
)
```

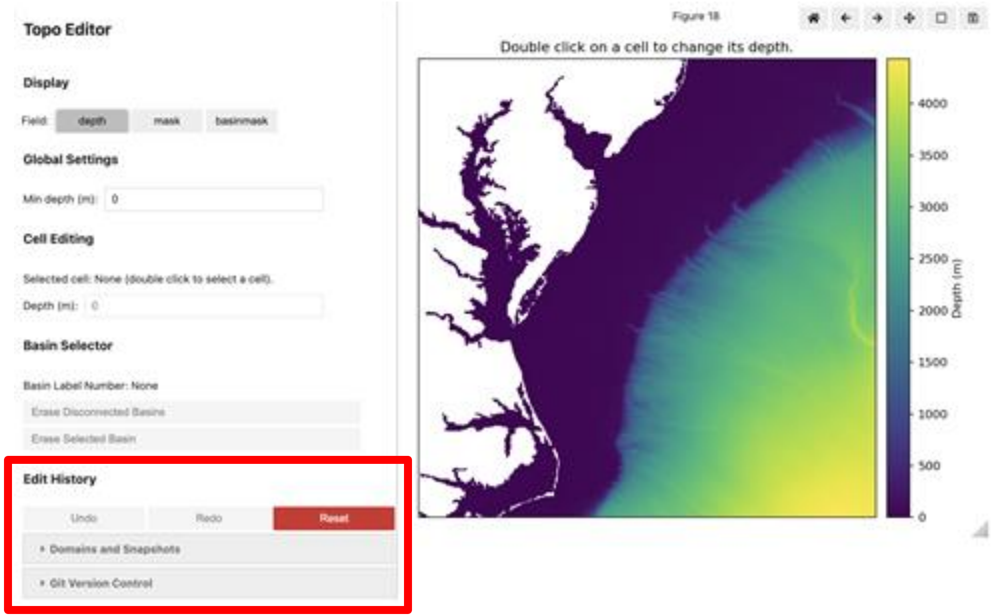
How do we make this workflow **reproducible**?

Reproducibility = Repeatability + Traceability + Transparency

History

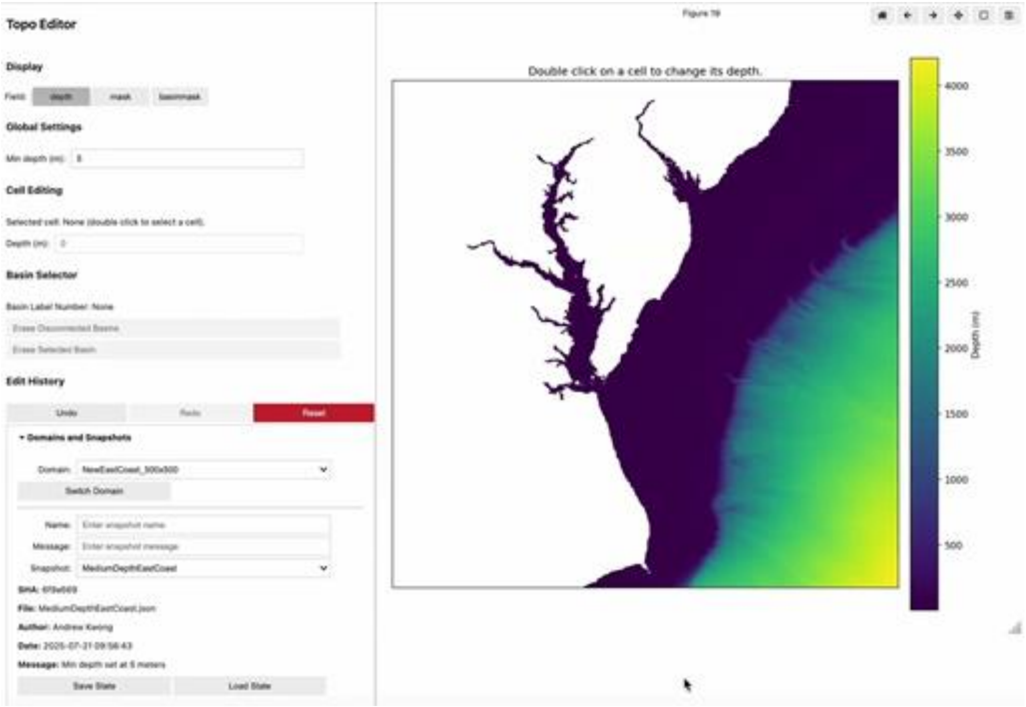
Version control

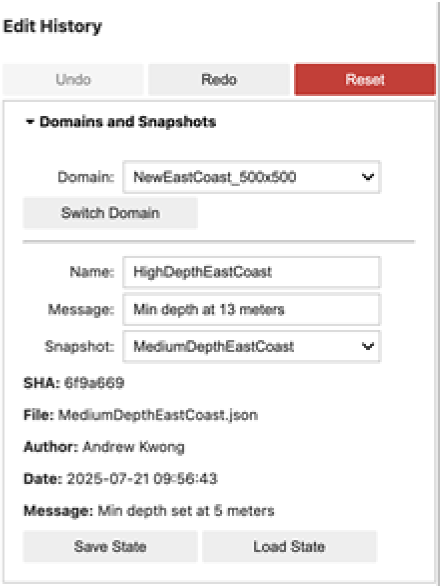
Shareability



Editing topography is important for model tuning but tedious!

HISTORY AND PROVENANCE





Topo Editor Edit History Section

Users now able to save and load snapshots across sessions, with a history **log** of modifications.

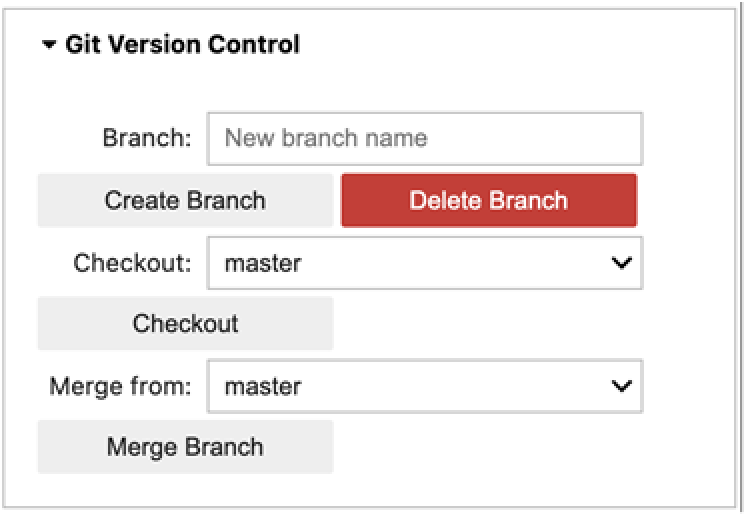
VERSION CONTROL INTEGRATION

With a history to log changes over time, how do we integrate version control?

We use **Git** to handle commits, branches, and checkouts in the Topo Editor.

This promotes data provenance and traceability!

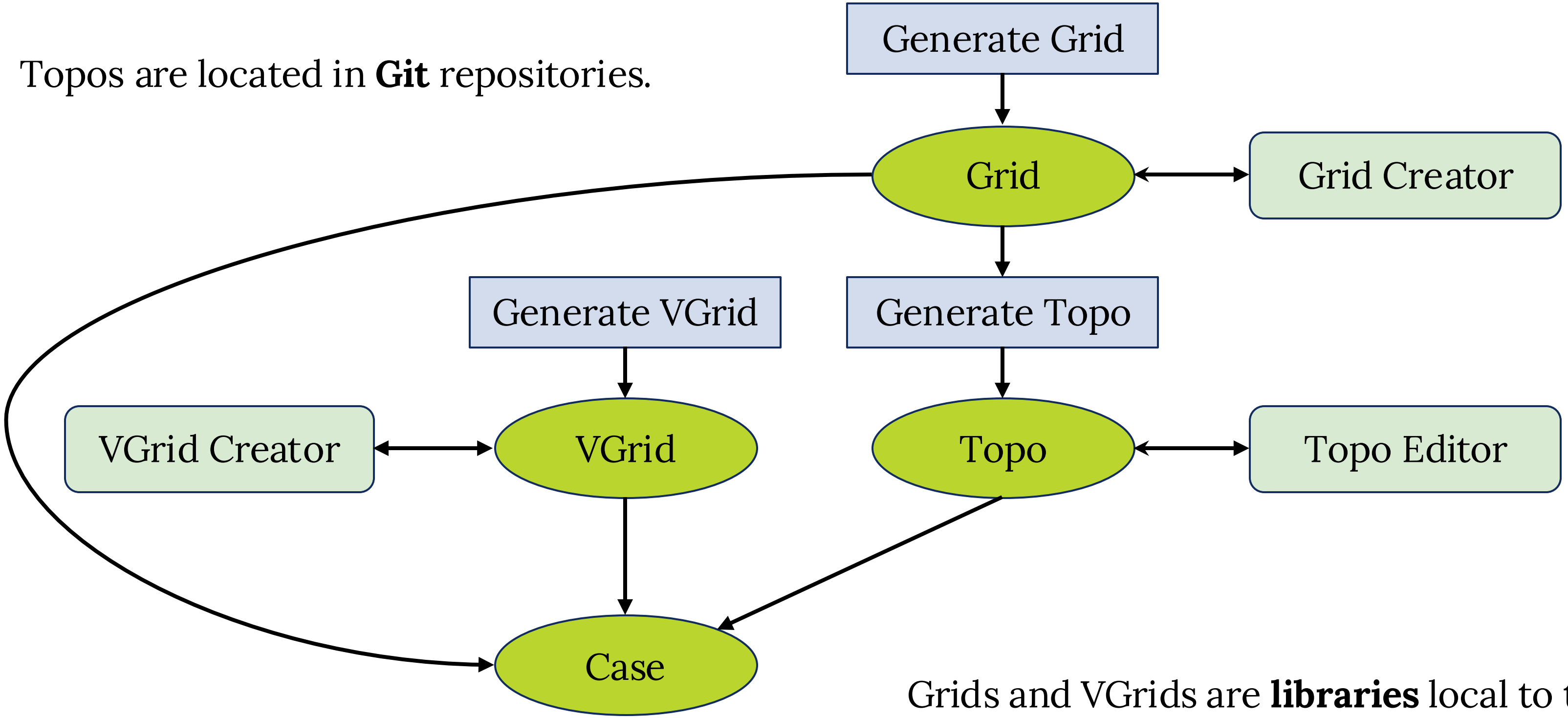
Users can share Topo and Grid configurations through repositories.



Topo Editor Version Control Section

STRUCTURE AND SHAREABILITY

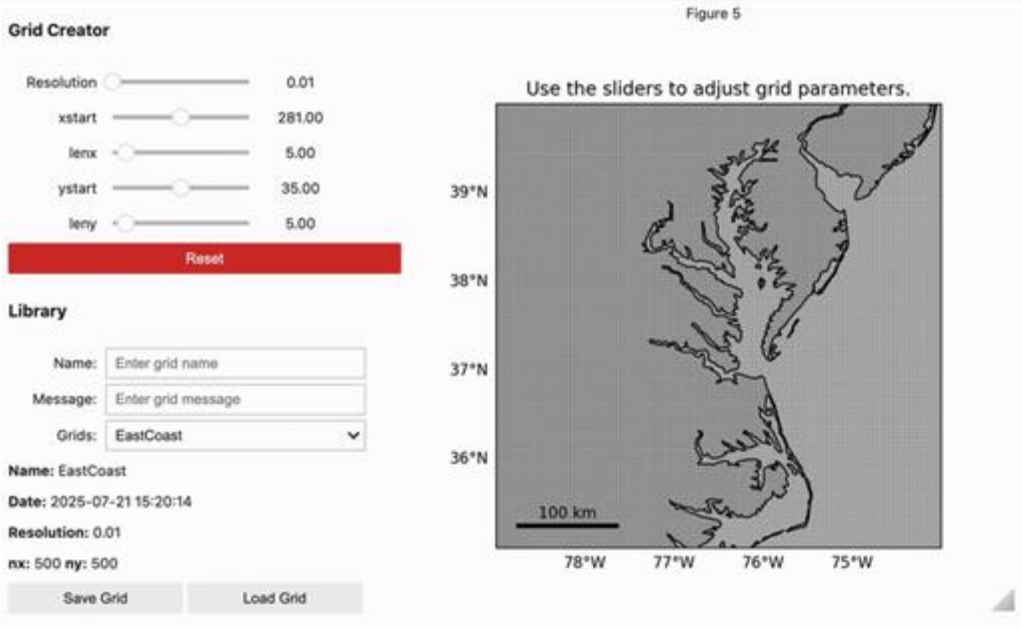
Topos are located in **Git** repositories.



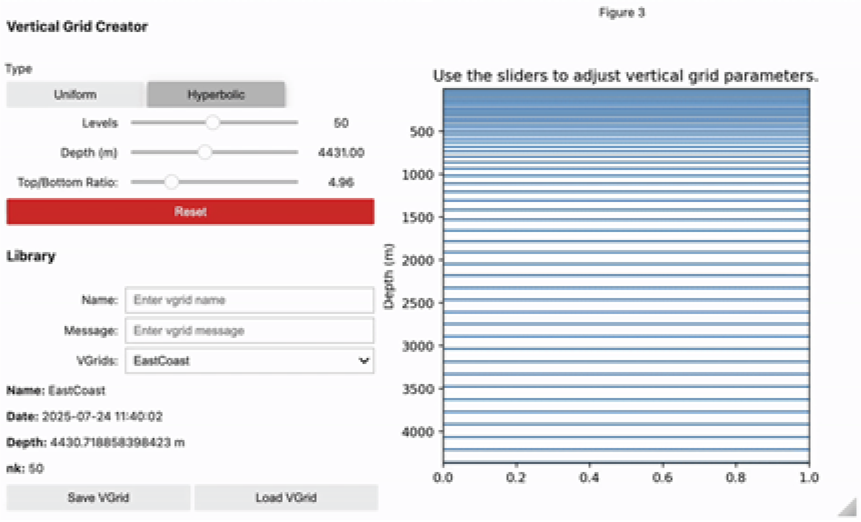
Grids and VGrids are **libraries** local to the user.

OBJECT CONFIGURATORS

Grid Creator



VGrid Creator

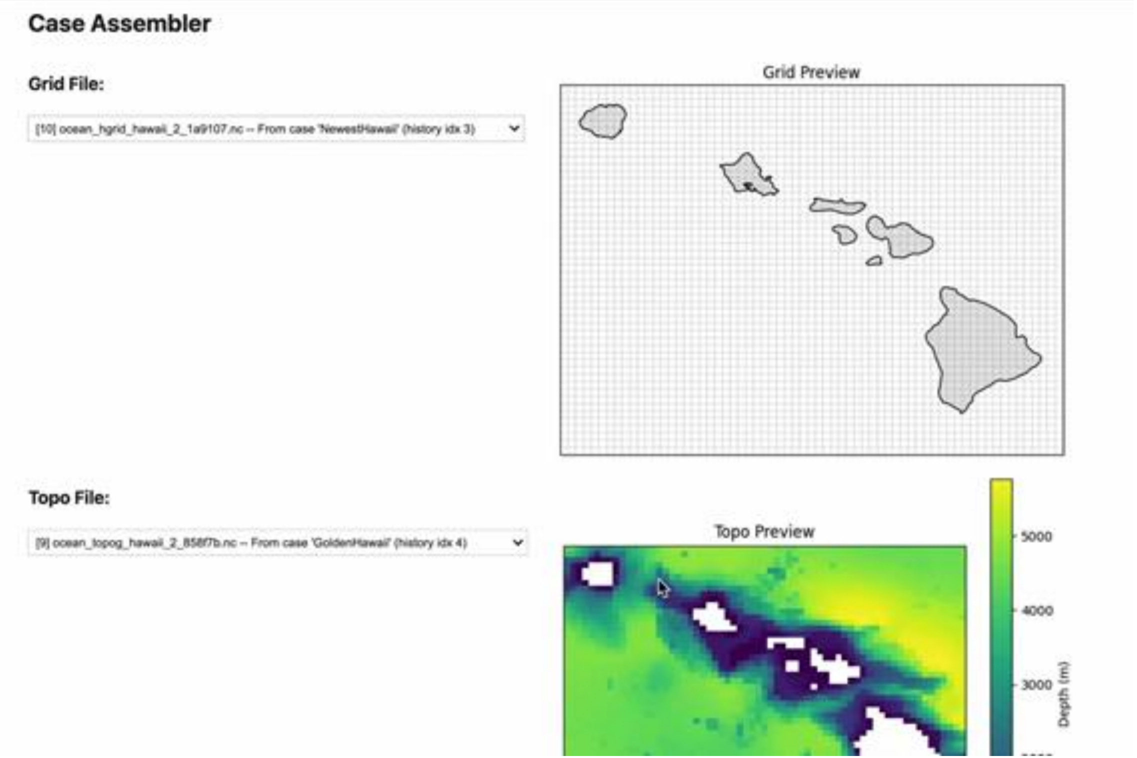


Case Assembler

Final step: Add reproducibility to the preparation of forcing data.

Case history configurations are saved to a .json file.

We have everything we need to **build** and **run** a Case from history!



Final step: Add reproducibility to the preparation of forcing data.

Case history configurations are saved to a .json file.

We have everything we need to **build** and **run** a Case from history!

```
Adding parameter changes to user_nl_mom:

  ! Open boundary conditions
  OBC_NUMBER_OF_SEGMENTS = 4
  OBC_FREESLIP_VORTICITY = False
  OBC_FREESLIP_STRAIN = False
  OBC_COMPUTED_VORTICITY = True
  OBC_COMPUTED_STRAIN = True
  OBC_ZERO_BIHARMONIC = True
  OBC_TRACER_RESERVOIR_LENGTH_SCALE_OUT = 3.0E+04
  OBC_TRACER_RESERVOIR_LENGTH_SCALE_IN = 3000.0
  BRUSHCUTTER_MODE = True
  OBC_TIDE_N_CONSTITUENTS = 0
  OBC_TIDE_CONSTITUENTS = ""
  OBC_SEGMENT_001 = "J=0,I=0:N,FLATHER,ORLANSKI,NUDGED,ORLANSKI_TAN,NUDGED_TAN"
  OBC_SEGMENT_001_VELOCITY_NUDGING_TIMESCALES = 0.3, 360.0
  OBC_SEGMENT_001_DATA = "U=file:forcing_obc_segment_001.nc(u),V=file:forcing_obc_segment_001.nc(v)
c(salt)"
  OBC_SEGMENT_002 = "J=N,I=N:0,FLATHER,ORLANSKI,NUDGED,ORLANSKI_TAN,NUDGED_TAN"
  OBC_SEGMENT_002_VELOCITY_NUDGING_TIMESCALES = 0.3, 360.0
  OBC_SEGMENT_002_DATA = "U=file:forcing_obc_segment_002.nc(u),V=file:forcing_obc_segment_002.nc(v)
c(salt)"
  OBC_SEGMENT_003 = "I=0,J=N:0,FLATHER,ORLANSKI,NUDGED,ORLANSKI_TAN,NUDGED_TAN"
  OBC_SEGMENT_003_VELOCITY_NUDGING_TIMESCALES = 0.3, 360.0
  OBC_SEGMENT_003_DATA = "U=file:forcing_obc_segment_003.nc(u),V=file:forcing_obc_segment_003.nc(v)
c(salt)"
  OBC_SEGMENT_004 = "I=N,J=0:N,FLATHER,ORLANSKI,NUDGED,ORLANSKI_TAN,NUDGED_TAN"
  OBC_SEGMENT_004_VELOCITY_NUDGING_TIMESCALES = 0.3, 360.0
  OBC_SEGMENT_004_DATA = "U=file:forcing_obc_segment_004.nc(u),V=file:forcing_obc_segment_004.nc(v)
c(salt)"

./xmlchange RUN_STARTDATE=2020-01-01 --non-local

./xmlchange MOM6_MEMORY_MODE=dynamic_symmetric --non-local

Case is ready to be built: /glade/u/home/akwong/croc_cases/VirginiaLowRun
Forcings configured and processed.
```

1. Reproducibility requires intention and infrastructure.

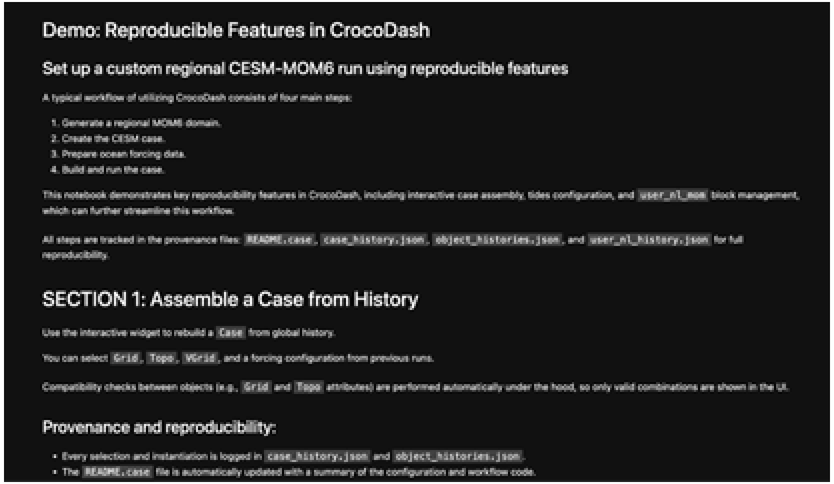
2. Modular design enables reusable modeling.

1. Good science begins with reproducibility.

Our work integrating modular configurators, version-controlled editing, and reproducible tools lays the foundation for rapid, reproducible Earth system modeling.

ADDITIONAL WORK

Documentation


Demo: Reproducible Features in CrocoDash

Set up a custom regional CESM-MOM6 run using reproducible features

A typical workflow of utilizing CrocoDash consists of four main steps:

1. Generate a regional MOM6 domain.
2. Create the CESM case.
3. Prepare ocean forcing data.
4. Build and run the case.

This notebook demonstrates key reproducibility features in CrocoDash, including interactive case assembly, tides configuration, and `user_nl_mom` block management, which can further streamline this workflow.

All steps are tracked in the provenance files: `README.case`, `case_history.json`, `object_histories.json`, and `user_nl_history.json` for full reproducibility.

SECTION 1: Assemble a Case from History

Use the interactive widget to rebuild a `Case` from global history.

You can select `Grid`, `Topo`, `VGrid`, and a forcing configuration from previous runs.

Compatibility checks between objects (e.g., `Grid` and `Topo` attributes) are performed automatically under the hood, so only valid combinations are shown in the UI.

Provenance and reproducibility:

- Every selection and instantiation is logged in `case_history.json` and `object_histories.json`.
- The `README.case` file is automatically updated with a summary of the configuration and workflow code.

Creation of Jupyter notebooks focused on scientific reproducibility for regional MOM6 cases.

Testing

- Write unit tests for key components in the editors and in the Case Assembler.
- Develop a continuous integration (CI) pipeline for automated testing.

Extending Features

- Handle for other grid and edit types.
- Add to the reproducibility suite as CrocoDash grows (e.g., chlorophyll, BGC, DROF)!

ACKNOWLEDGMENTS

Thank you to my mentors:

Dan Amrhein, Manish Venumuddula, Alper Altuntas

Thank you to the internship organizers:

Virginia Do, Jessica Wang, Prashansa Agrawal

Thank you to the organizations that support us:

This material is based upon work supported by the U.S. National Science Foundation National Center for Atmospheric Research, which is a major facility sponsored by the U.S. National Science Foundation under Cooperative Agreement No. 1755088. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the U.S. National Science Foundation.